University of Al-Maaqal

College of Engineering

Department of Artificial Intelligence Engineering

Predicting 10-Year Heart Disease Risk on the Framingham Data: Lasso Feature Selection and Model Comparison

Students Name:

Zainab Dakhel

Farah Hamid

Supervised by:

Prof.Dr.Ali Gamal Rabee

1. The Dataset

1.1-Overview

The project uses the **Framingham Heart Study** dataset, a long-running cardiovascular study of residents of Framingham, Massachusetts. Each row is one person, described by a routine check-up (age, blood pressure, cholesterol, smoking habits, and so on). The goal is to predict whether that person develops **coronary heart disease (CHD) within the next 10 years**. This is a binary classification problem. The data was taken from the public Kaggle version of the dataset (file framingham.csv).

- **Size:** 4,238 patients and 16 columns (15 predictors and 1 target).
- **Task:** predict the target TenYearCHD (1 = developed CHD within 10 years, 0 = did not).
- **Why it is hard:** the outcome lies 10 years in the future, so the check-up values carry only a weak signal, and only 15% of patients are positive.

1.2 Description and types of dataset features

Variable	Type	Description
male	binary	1 = male, 0 = female
age	numeric	Age in years
education	Categorical(1 to 4)	Education level
currentSmoker	binary	Currently smokes
cigsPerDay	numeric	Cigarettes smoked per day
BPMeds	binary	Takes blood pressure medication
prevalentStroke	binary	Has had a stroke
prevalentHyp	binary	Has hypertension
Diabetes	binary	binary
totChol	numeric	Total cholesterol
sysBP	numeric	Systolic blood pressure
diaBP	numeric	Diastolic blood pressure
BMI	numeric	Body mass index
heartRate	numeric	Heart rate
glucose	numeric	Blood glucose
TenYearCHD	**Binary(target)**	**Developed CHD within 10 years**

1.3 Class balance

Of the 4,238 patients, **3,594 (84.8%)** did not develop CHD and **644 (15.2%)** did. The data is therefore imbalanced: a model that says "nobody gets sick" would be 84.8% accurate while detecting no patient at all. For this reason accuracy alone is not a reliable measure here, and the project reports Recall, Precision, F1 and ROC-AUC as well.

1.4 Missing values

Variable	Missing	Share
glucose	388	9.16%
education	105	2.48%
BPMeds	53	1.25%
totChol	50	1.18%
cigsPerDay	29	0.68%
BMI	19	0.45%
heartRate	1	0.02%

All shares are small, so no rows were deleted. The values are filled later, inside the pipeline (Section 3.4)

2. Methodology: Step-by-Step Workflow

The workflow follows the order requested by the instructor, with one methodological refinement: the train/test split is done before any step that learns from the data."

1- **Visualization and EDA**. Look at the target distribution, histograms, boxplots against the target, and a correlation heatmap. These plots only inspect the data and change nothing, so they can be done before the split.
2- **Train/test split (80/20, stratified).** The test set (848 patients) is set aside and not touched until the end. Stratification keeps the 15.2% CHD share in both parts
3- **Missing-value imputation**. Median for continuous variables, most frequent value for binary and categorical ones. Learned from the training set only.
4- **Distribution handling**. glucose is strongly skewed (skewness 6.21), so a log1p transform is applied.
5- **Feature scaling.** StandardScaler for the continuous variables, learned from the training set only.

6- **Dimensionality reduction with Lasso.** A logistic regression with an L1 penalty sets the coefficients of weak variables to exactly zero, and only the selected variables are passed on. PCA was not used, following the instructor's advice (the data contains many binary variables).
7- **Model training**. The selected variables are passed to Logistic Regression, Random Forest and XGBoost, all trained on the training set.
8- **Testing and evaluation**. Each model is tested once on the test set and compared with Accuracy, Recall, Precision, F1, ROC-AUC and the confusion matrix.
9- **Model choice and interpretation**. The best model is chosen and its coefficients are interpreted.

Why the split comes first ?

Imputation, scaling and Lasso all compute statistics from data (medians, means, which variables to keep). If they see all rows before the split, information from the test set leaks into training (**data leakage**) and the final score looks better than it really is. All learned steps are therefore placed inside one scikit-learn Pipeline, which fits them on the training set only

Split → Impute → log1p (glucose) → Scale → Lasso selection → Model
(steps 2 to 6 are fitted on the training set only)

3. Code Explanation

The whole project is one notebook, Heart_Disease_Full_Project.ipynb. It runs the shared steps once (loading, EDA, split, preprocessing, Lasso) and then trains and compares three models with the same pipeline: Logistic Regression (the main model), Random Forest and XGBoost. Each part below explains one block of the code.

3.1 Libraries and loading the data

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns

df = pd.read_csv("framingham.csv")
print(df.shape) # (4238, 16)
df.info()
```

The file is loaded into a pandas DataFrame, and shape and info() confirm 4,238 rows, 16 columns and the missing values listed in Section 1.4.

3.2 Exploratory data analysis

```
sns.countplot(x="TenYearCHD", data=df)

 # class balance
df[cont].hist(bins=30, figsize=(14, 8))

# distributions
print(df[cont].skew().round(2))

# skewness
sns.boxplot(x="TenYearCHD", y=col, data=df)

# per variable
sns.heatmap(df.corr(), annot=True,

cmap="coolwarm")
```

What the plots show:

• **Target distribution:** 84.8% vs 15.2%, so the data is imbalanced.

• **Histograms and skewness:** glucose is the most skewed variable (6.21); cigsPerDay (1.25) and sysBP (1.15) are moderately skewed; the others are close to normal. Only glucose receives a log transform.

• **Boxplots:** patients who developed CHD are older and have higher blood pressure. Many outliers exist in glucose, sysBP and totChol; they were **kept** because they are real clinical values.

• **Heatmap:** the strongest correlations are sysBP/diaBP (0.78), currentSmoker/cigsPerDay (0.77), prevalentHyp/sysBP (0.70) and diabetes/glucose (0.62). This overlap (multicollinearity) is a reason to use Lasso. The highest correlation with the target is only 0.23 (age), followed by sysBP at 0.22, so any model will have a modest ceiling.

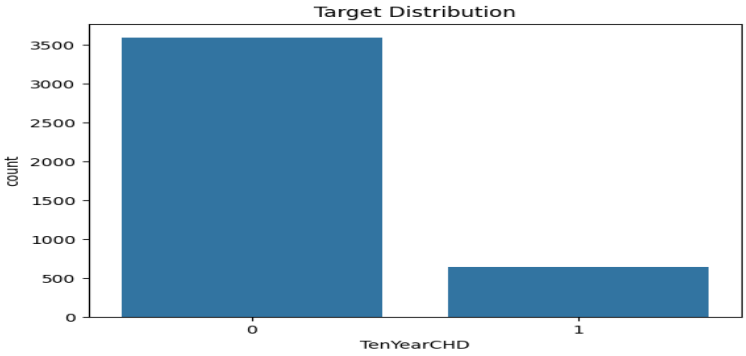


Figure 1: target distribution bar chart

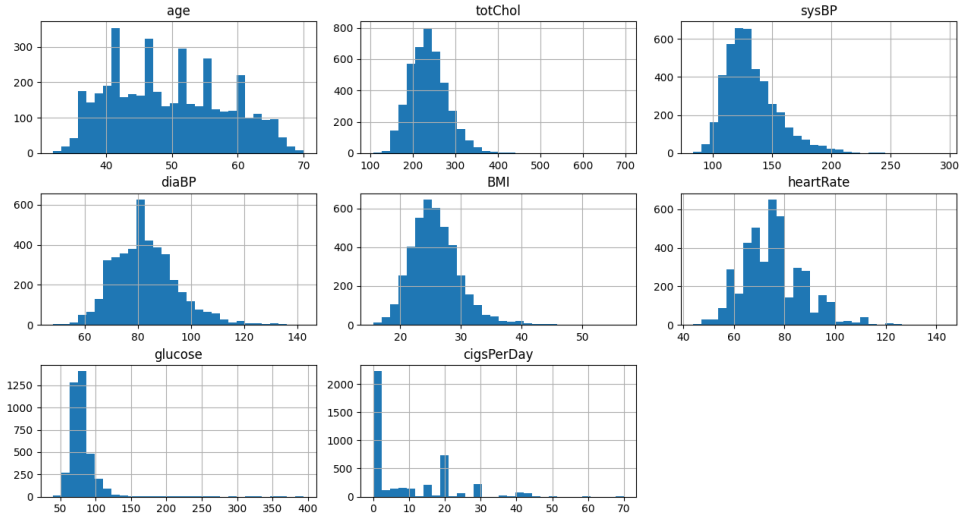


Figure 2: histograms of the continuous variables

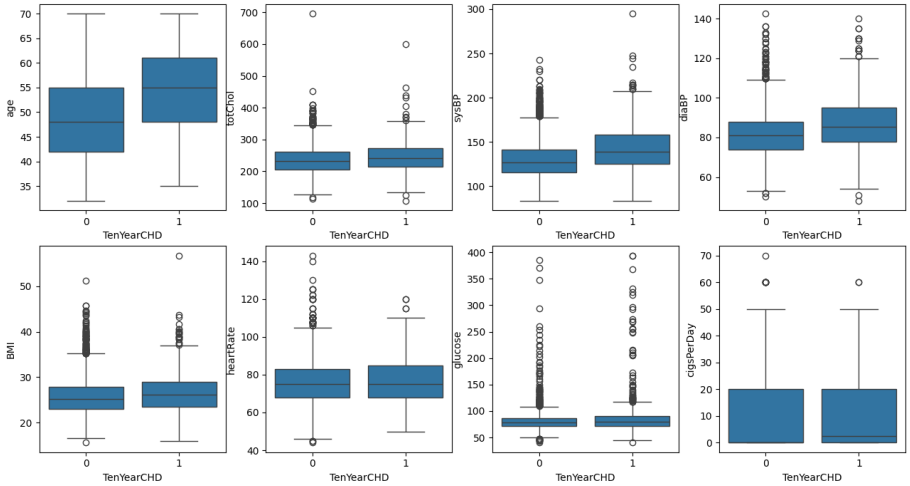


Figure 3: boxplots against TenYearCHD

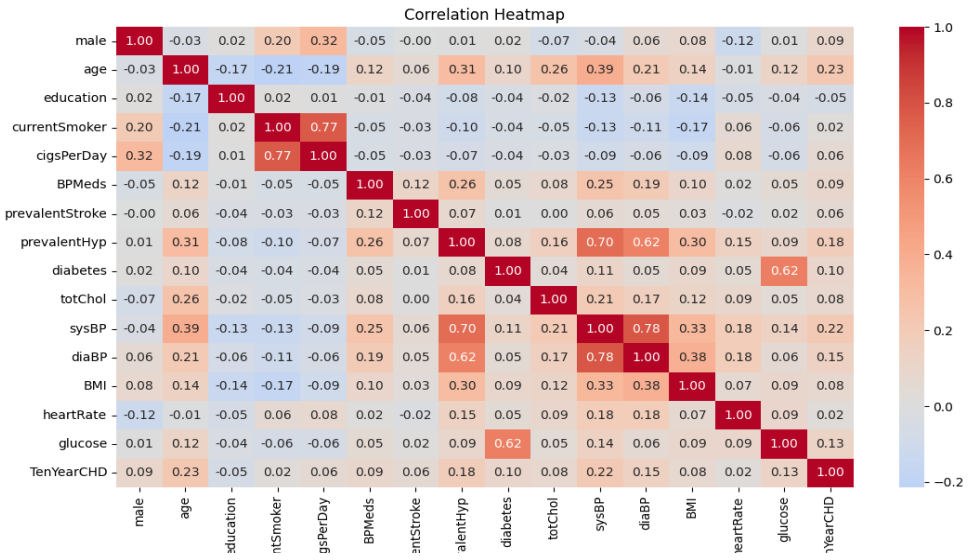


Figure 4: correlation heatmap

3.3 Train/test split

```
X = df.drop(columns=["TenYearCHD"])
y = df["TenYearCHD"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, stratify=y, random_state=42)
```

The data is split into 3,390 training and 848 test patients. stratify=y keeps the class ratio (0.848 / 0.152) in both parts, and random_state=42 makes the split reproducible. The test set contains 129 CHD patients.

3.4 Preprocessing inside a pipeline

```
log_cols =  ["glucose"]
num_cols = ["age","cigsPerDay","totChol","sysBP","diaBP","BMI","heartRate"]
cat_cols = ["male","education","currentSmoker","BPMeds",
"prevalentStroke","prevalentHyp","diabetes"]

log_pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
("log", FunctionTransformer(np.log1p)),
("scaler", StandardScaler())])
num_pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
("scaler", StandardScaler())])
cat_pipe = Pipeline([("imputer", SimpleImputer(strategy="most_frequent"))])

preprocessor = ColumnTransformer([("log", log_pipe, log_cols),
("num", num_pipe, num_cols),
("cat", cat_pipe, cat_cols)])
```

The ColumnTransformer applies a different treatment to each group of columns. glucose is imputed, log-transformed and scaled. The other continuous variables are imputed with the median and scaled. Binary and categorical variables are imputed with the most frequent value and are not scaled. When fit is called on the training set, all medians, means and standard deviations are learned from the training data only, and the test data is only transformed. A check after fit_transform confirmed that no missing values remain in either set

3.5 Lasso as dimensionality reduction

```
selector = SelectFromModel(
    LogisticRegression(penalty="l1", solver="liblinear",
C=0.1, class_weight="balanced", random_state=42))
```

The L1 (Lasso) penalty pushes the coefficients of weak variables to exactly zero. SelectFromModel keeps only variables with a non-zero coefficient. C controls the strength of the penalty (smaller means stronger). With C=1.0 no variable was removed, so C=0.1 was used, which removed **4 of 15 variables**: heartRate, currentSmoker, prevalentStroke and diabetes. currentSmoker repeats the information in cigsPerDay (correlation 0.77). prevalentStroke is rare in the data, so a strong penalty removes it; this does not mean it is clinically irrelevant. class_weight="balanced" gives more weight to the rare CHD class.

variable	Lasso coefficients
age	0.590
male	0.329
sysBP	0.297
cigsPerDay	0.291
glucose	0.125
BPMeds	0.117
prevalentHyp	0.097
totChol	0.084
education	-0.038
diaBP	0.014
BMI	0.013
heartRate, currentSmoker,prevalentStrke, diabetes	0(removed)

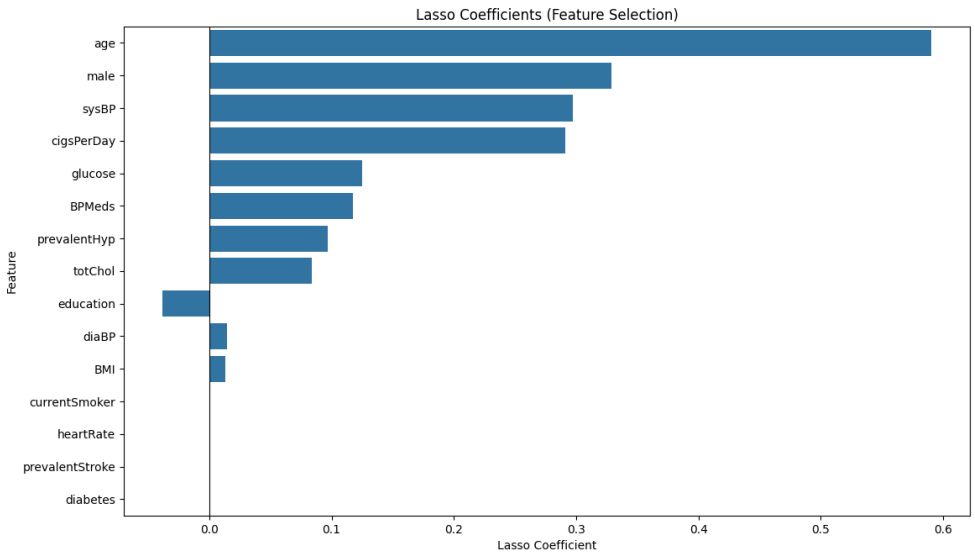


Figure 5: Lasso coefficients bar chart

3.6 Building and training the models

```
def build(clf):
return Pipeline([
("prep", clone(preprocessor)), # imputation + log + scaling
("select", clone(selector)), # Lasso keeps the important variables
("clf", clf),
])
models = {
"Lasso + Logistic": build(
LogisticRegression(class_weight="balanced", random_state=42)),
"Lasso + RandomForest": build(
RandomForestClassifier(n_estimators=300, max_depth=5, min_samples_leaf=10,
class_weight="balanced_subsample", random_state=42)),
"Lasso + XGBoost": build(
XGBClassifier(n_estimators=300, max_depth=3, learning_rate=0.05,
subsample=0.8, colsample_bytree=0.8, scale_pos_weight=5.6,
eval_metric="logloss", random_state=42)),
}
for name, model in models.items():
model.fit(X_train, y_train)
```

The pipeline passes the data through preprocessing and Lasso, and **only the selected variables reach the final classifier**. Three classifiers were tried after the same Lasso step:

•**Logistic Regression** (main model): simple, interpretable, class_weight="balanced".
•**Random Forest:** 300 trees, max_depth=5, min_samples_leaf=10, class_weight="balanced_subsample". The depth limits prevent memorizing the data.
•**XGBoost:** 300 trees, max_depth=3, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, scale_pos_weight=5.6 (about 3,594 / 644, to handle the imbalance).

All three use the same split and the same random seed, so the comparison is fair.

3.7 Evaluation

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

accuracy_score(y_test, y_pred); recall_score(y_test, y_pred)
precision_score(y_test, y_pred); f1_score(y_test, y_pred)
roc_auc_score(y_test, y_prob); confusion_matrix(y_test, y_pred)
```

- **Accuracy:** share of all patients classified correctly (misleading with 85% healthy patients).
- **Recall:** share of true CHD patients that the model detects.
- **Precision:** share of the model's CHD alerts that are correct.
- **F1:** balance between Recall and Precision.
- **ROC-AUC:** how well the model ranks patients by risk, independent of the decision threshold (0.5 is random, 1.0 is perfect).

Performance metrics of Lasso + Logistic Regression model:

Model	Accuracy	Recall	Precision	F1	ROC-AUC
Lasso+Logistic	0.659	0.581	0.242	0.342	0.696

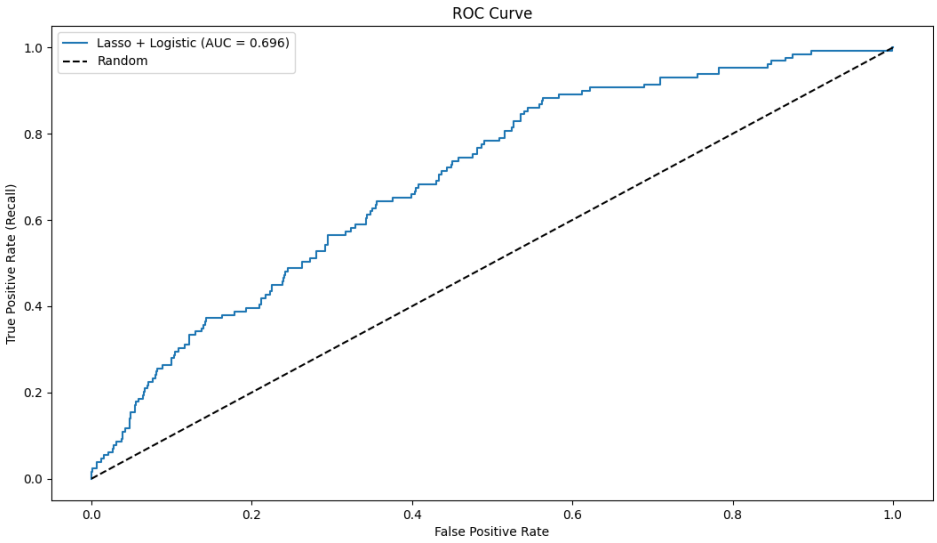


Figure 6:ROC curve(AUC=0.696)

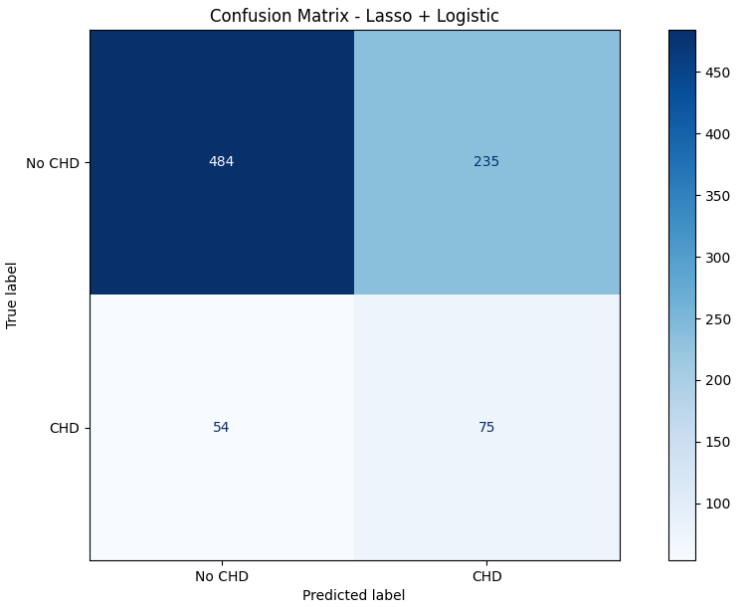


Figure 7: confusion matrix of Lasso+ Logistic Regression

3.8 Interpreting the final model

After Lasso, the Logistic Regression coefficients show that **age (0.606), BPMeds (0.492), male (0.420), cigsPerDay (0.303) and sysBP (0.274)** have the largest effect on risk. A positive coefficient raises the estimated risk. diaBP has a small coefficient because sysBP (correlation 0.78) already carries most of its information. The tree models agree that age and blood pressure matter (Random Forest: age 0.313, sysBP 0.183; XGBoost: prevalentHyp 0.198, age 0.163). Coefficients show association, not cause.

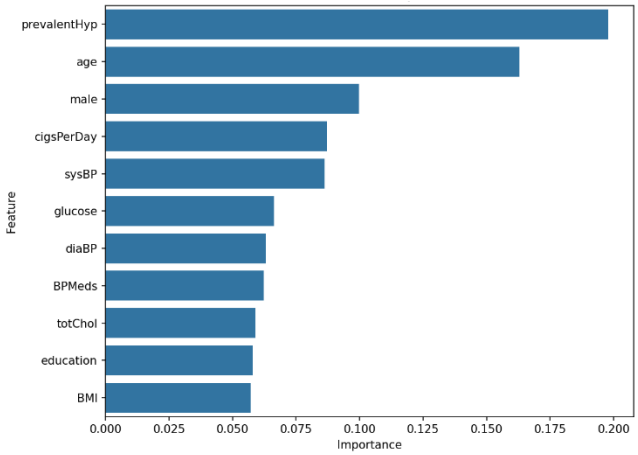


Figure 8: feature importance of XGBoost

3.9 Effect of the decision threshold

A model outputs a probability, and the threshold decides when a patient is labelled "CHD" (default 0.50). The best threshold for F1 was searched on the training set only (5-fold cross-validation), giving 0.58, then applied once to the test set (Logistic Regression only).

Threshold	Recall	Precision	F1	Accuracy	ROC-AUC
0.50(default)	0.581	**0.242**	**0.342**	**0.659**	**0.696**
0.58(tuned)	0.426	**0.258**	**0.322**	**0.726**	**0.696**

Raising the threshold raised Accuracy and Precision but lowered Recall and F1, and the AUC did not change. This is a trade-off, not an improvement. Accuracy rises only because most patients are healthy and fewer alarms are raised. For screening, where missing a patient is worse than a false alarm, the default 0.50 is kept.

4. Results and Model Comparison

Model	Accuracy	Recall	Precision	F1	ROC-AUC
Lasso+Logistic	0.659	0.581	0.242	0.342	0.696
Lasso+Random Forest	0.682	0.504	0.240	0.325	0.683
Lasso+XGBoost	0.676	0.481	0.230	0.311	0.660

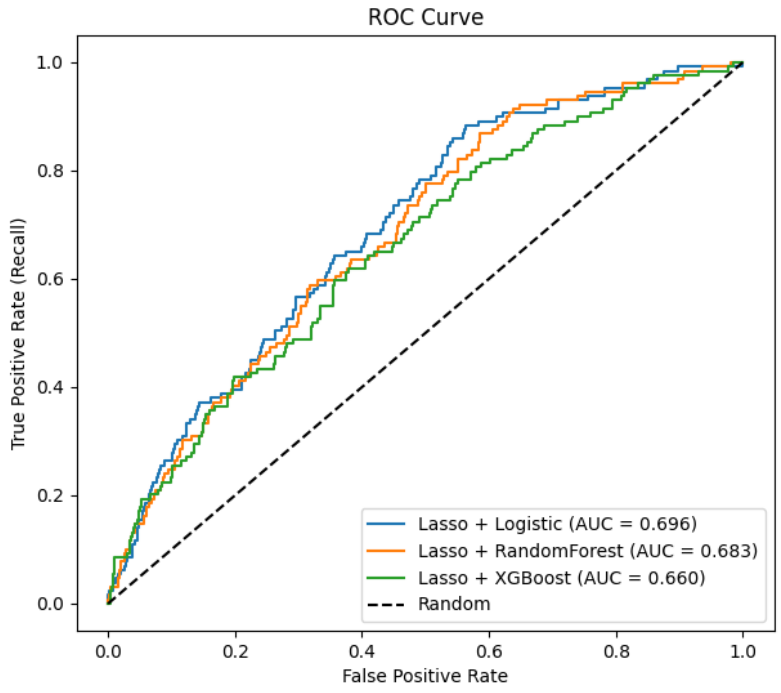


Figure 9: ROC curve of the models

Computed from the confusion matrix, (511 + 62) / 848. Confusion matrix of the final model (threshold 0.50): 484 healthy patients correctly classified, 235 false alarms, 54 missed CHD patients, 75 detected CHD patients. Logistic Regression gives the best AUC, Recall and F1 and is the easiest to interpret. Random Forest has the highest accuracy only because it raises fewer alarms (lower Recall). XGBoost has the lowest AUC. The three models perform similarly (AUC 0.66 to 0.70), so the limit comes from the data, not from the model.

5.Limitations

• **Moderate performance.** AUC about 0.70 is typical for this dataset. Precision is 0.24, so about three of four alerts are false. The model suits screening, not diagnosis.
• **C was chosen after seeing test results.** C=0.1 was used after C=1.0 removed no variable.Read it as "same performance with fewer variables", not as proof of an optimal value.

• **Small test set and single split.** Only 129 CHD cases are in the test set, so small differences between models are within the margin of error.
• **Association, not causation.** Coefficients describe patterns in this cohort and cannot prove that a factor causes disease**.**

6. Conclusion

A leakage-free pipeline was built to predict 10-year heart disease risk: split first, then imputation, log transform, scaling, Lasso feature selection and a classifier. Lasso (C=0.1) removed four variables without hurting performance. **Lasso + Logistic Regression** gave the best AUC (0.696), Recall (0.581) and F1 (0.342) and is the most interpretable, so it is the final model. The strongest risk factors were age, sex, systolic blood pressure and cigarettes per day.

7.References

Kaggle Dataset Link